

Video streaming

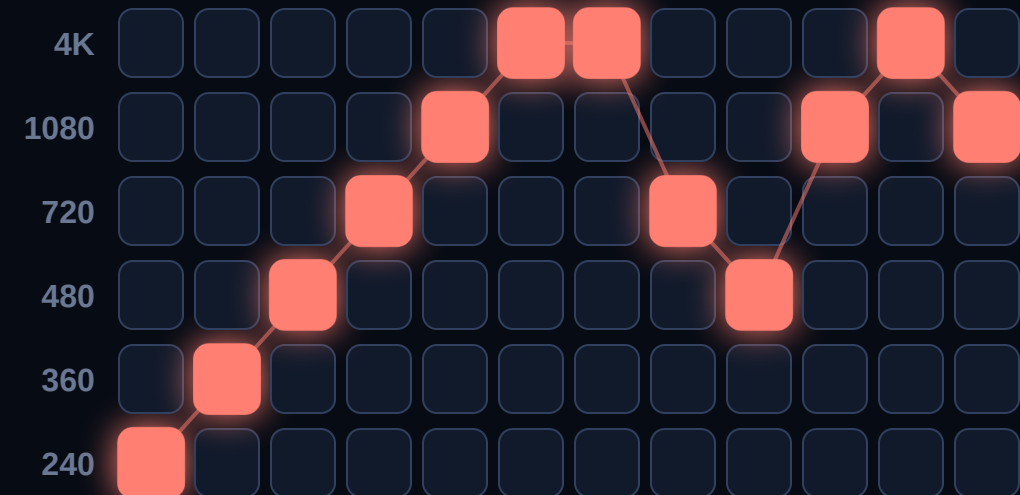
Segments, buffers, and the bitrate you get

Video is most of the traffic on the internet. This lecture explains how it is delivered: how it is compressed, how it is cut into segments, how the player keeps a buffer of them, how it picks a quality for each one, and where the segments are stored.

Yaxiong Xie

ONE VIDEO

twelve segments, six qualities



BUFFER

18 s downloaded

not downloaded

Why does video need its own lecture?

01

Why video

Video is large, it must arrive on time, and its size can be changed. Those three facts, plus a network whose speed keeps changing, are the whole problem.

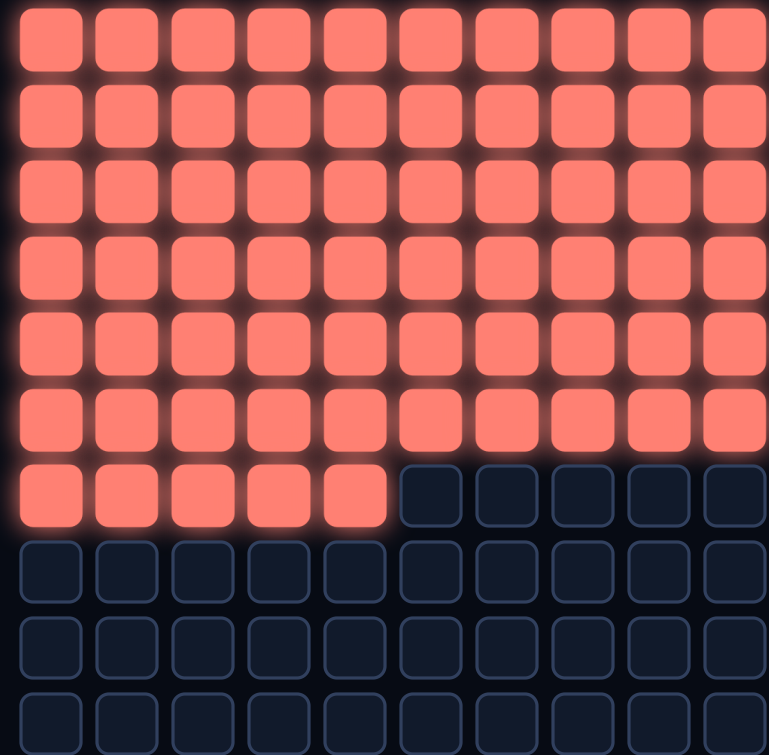
Video is about 65% of all internet traffic

65%

of all internet traffic is video

It was 51% in 2016. Measured by Sandvine.

The other 35% is everything else:
web, email, updates, games, file transfer, chat.



ONE SQUARE = 1% OF INTERNET TRAFFIC

Most of the bytes moving on the internet are video.

1 Why video

2 Encoding

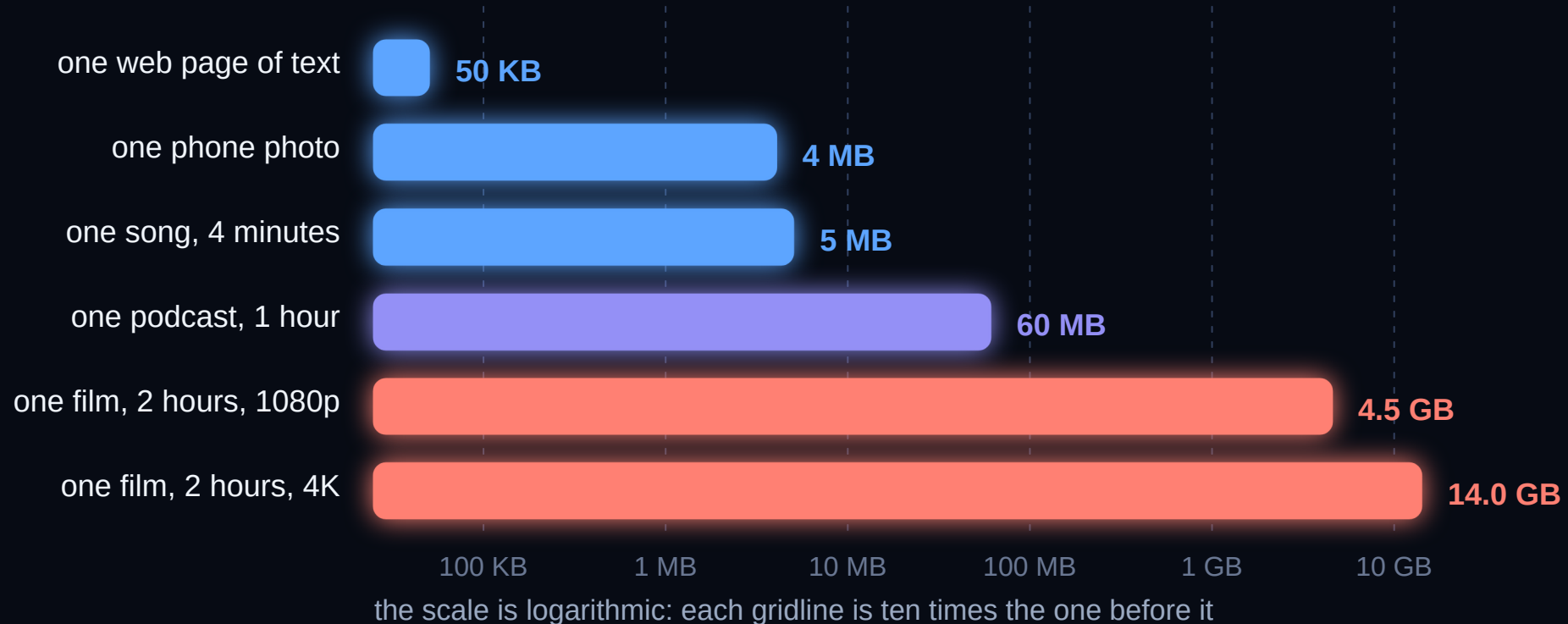
3 Segments

4 The player

5 Which row

6 CDN

A two-hour film is about 250,000 times a web page



Nothing else an ordinary user downloads is close to this size. That is why delivering video needs a design of its own.

A film is the largest object an ordinary user downloads.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

A film needs a moderate speed held for two hours

HOW MUCH BANDWIDTH EACH ONE NEEDS, AND FOR HOW LONG

a web page



20 Mbps for 0.4 seconds, then nothing

a 1080p film



720p

3 Mbps

1080p

5 Mbps

4K

15 Mbps

Netflix publishes these as the minimum connection speed for each quality.

A web page uses the whole connection for a fraction of a second and then gives it back. A film asks for a lower speed but holds it for two hours.

Video needs a steady rate over a long time, not a fast burst.

1 Why video

2 Encoding

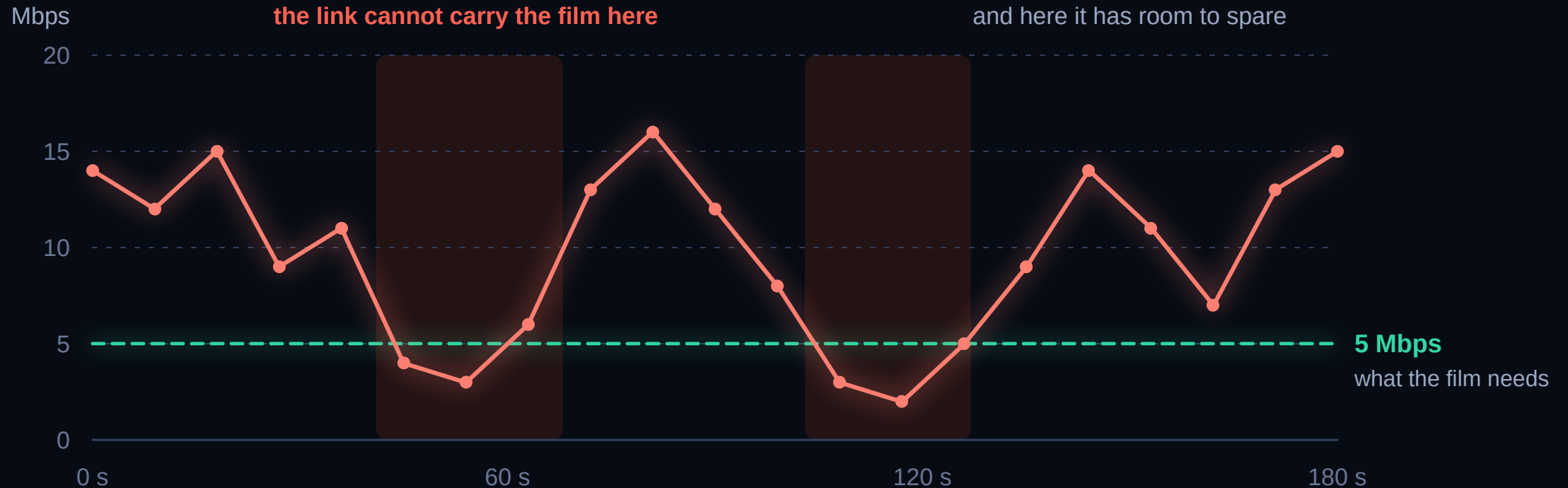
3 Segments

4 The player

5 Which row

6 CDN

But the speed of the connection does not hold still



One home Wi-Fi connection, measured for three minutes. It is faster than the film needs most of the time and slower than it twice.
Nothing in the network promises otherwise.

The player has to work with whatever the link gives it at that moment.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

Three facts about video, and only video has all three

It is large

4.5 GB for one 1080p film
about 250,000 web pages
the transfer takes minutes

size

It has a deadline

the video for second 37
must be there at second 37
late means the video stops

time

Its size is adjustable

one second of video can be
0.04 MB or 1.9 MB
the player chooses which

quality

A file download is large but has no deadline. A phone call has a deadline but is small. Only video is large, timed, and adjustable at the same time.

The third fact is what the rest of the lecture uses.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

Three requirements follow from those three facts

1

Start playing before the download finishes

the viewer should wait seconds, not minutes

2

Change quality while the video is playing

so the player can follow a link whose speed keeps changing

3

Download only the part that is watched

stop watching at minute 3 and pay for 3 minutes

The rest of the lecture is the design that meets all three, and the one decision that design leaves the player to make on its own.

Nothing that treats the video as a single file can do all three.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

Six parts

- | | | |
|----|---------------------------|--|
| 01 | Why video | size, timing, and a link that keeps changing |
| 02 | How video is encoded | frames, differences, and what a bitrate is |
| 03 | How it is cut up | segments, the ladder, and the manifest |
| 04 | What the player does | the buffer, and what empties it |
| 05 | Which row of the ladder | throughput, buffer level, the decision |
| 06 | Where the bytes come from | caches inside your internet provider |

Parts 2 and 3 are about the file the server holds. Parts 4 and 5 are about what the player does with it. Part 6 is about where it is stored.

One question runs through all six: how do you play what you have not downloaded?

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

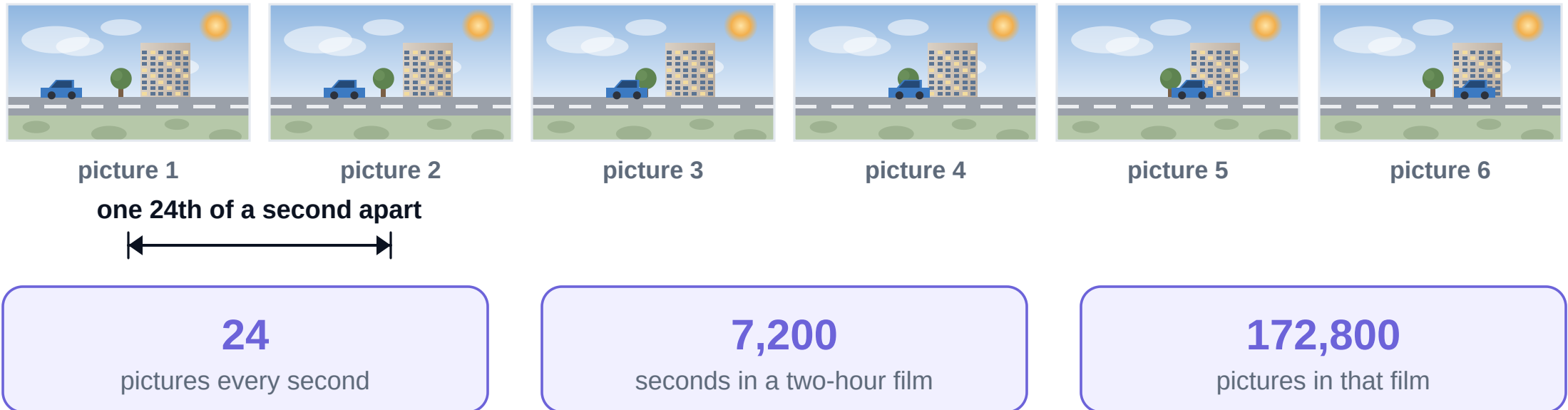
What does the encoder actually produce?

02

How video is encoded

A video is a sequence of pictures, and sending them as they are would need a gigabit link. This part is how they are made small, what is lost doing it, and what a bitrate is.

A video is a sequence of still pictures



One picture is called a frame, and how many are shown per second is the frame rate. Film uses 24; television and most web video use 25, 30 or 60.

A video file is a list of pictures, shown fast enough to look like motion.

1 Why video

2 Encoding

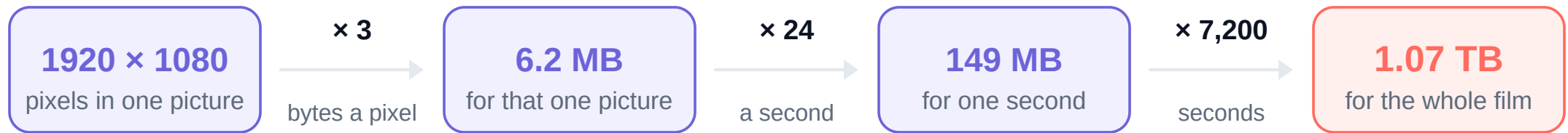
3 Segments

4 The player

5 Which row

6 CDN

Every picture is large, and there are a lot of them



WHAT IS ACTUALLY SENT TO YOU



So how does the encoder make it 240 times smaller?

the rest of this part is the answer

Each pixel needs three bytes, one for red, one for green and one for blue. At that size the film would need a 1.2 Gbps link to play in real time.

Without compression there is no video on the internet.

1 Why video

2 Encoding

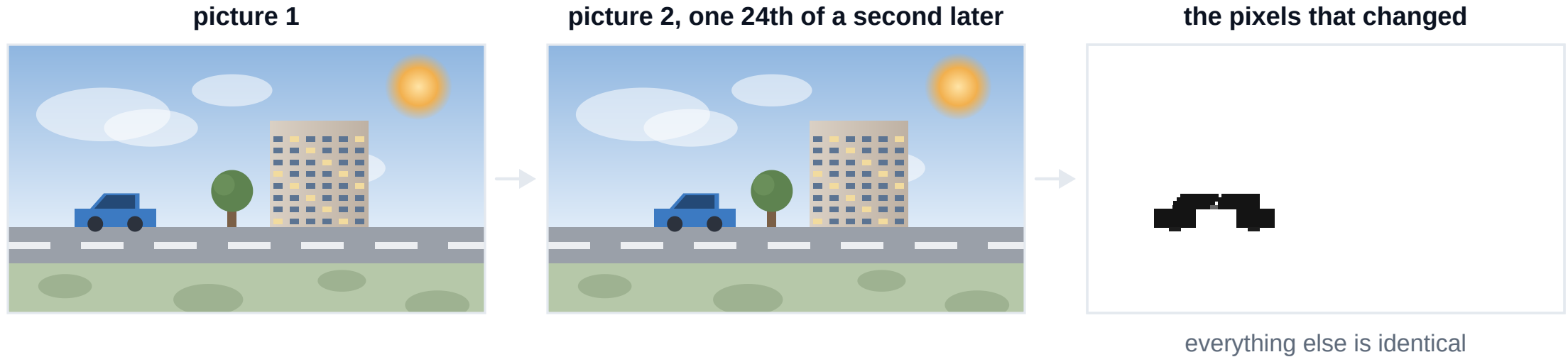
3 Segments

4 The player

5 Which row

6 CDN

Two pictures in a row are almost the same picture



so store what changed, not the whole picture again

this is where almost all of the 240× comes from

Twenty-four times a second, most of the screen does not change. An encoder that sent every picture on its own would send the same sky and the same road 172,800 times.

So store one picture, then store only what changed in the next ones.

1 Why video

2 Encoding

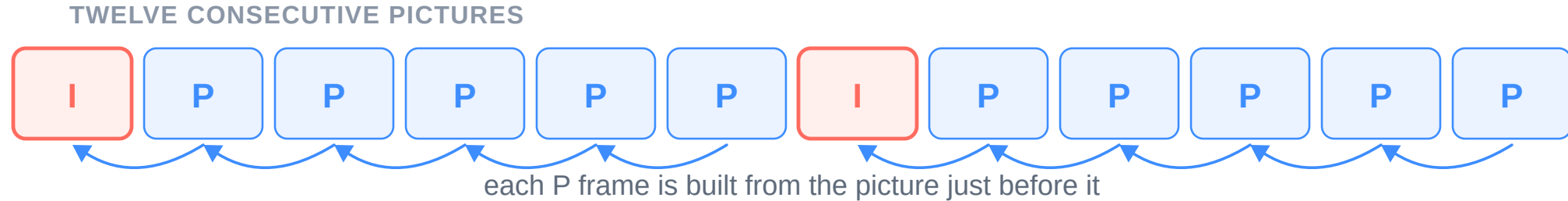
3 Segments

4 The player

5 Which row

6 CDN

Store one full picture, then only what changed



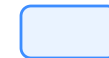
the whole picture, stored on its own



the most bits



only what changed since the picture before it



a small fraction of an I frame

a decoder can only start at an I frame

To show a P frame you must first have decoded the one before it, and the one before that, back to the nearest I frame. How often an I frame appears is a setting; half a second is common for television.

Where the I frames are is where the video can be cut. Part 3 uses this.

1 Why video

2 Encoding

3 Segments

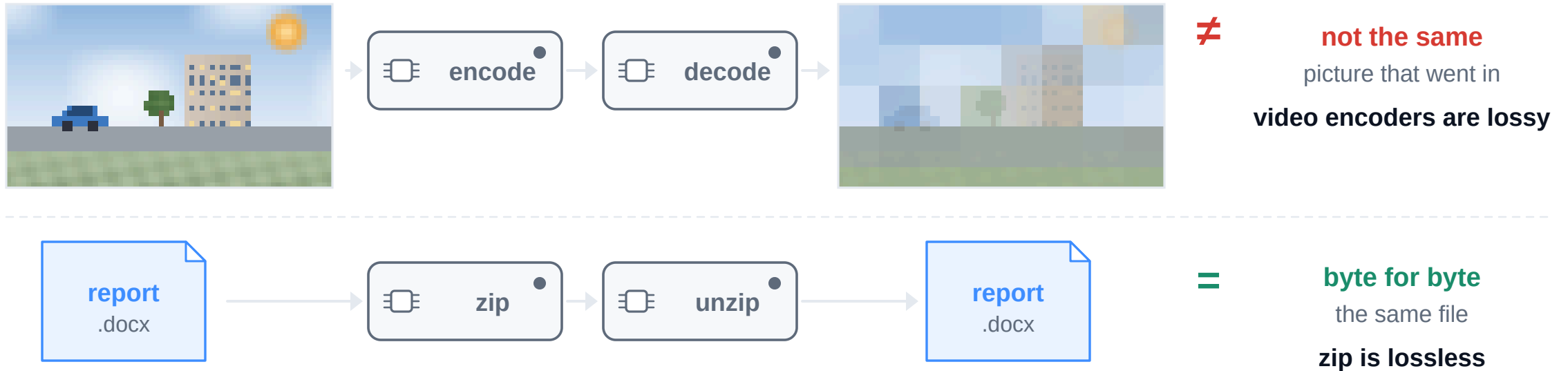
4 The player

5 Which row

6 CDN

- Compression throws information away

What comes out is not what went in



A zip file gives back exactly what went in. A video encoder does not, and that is how it reaches 240x. How much it throws away is a setting on the encoder.

Smaller and better are the two ends of one dial.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

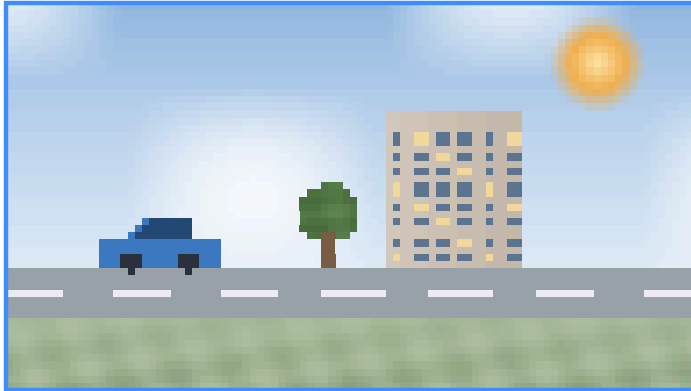
6 CDN

• — A second idea

Use fewer pixels for every picture

how many pixels a picture is made of is called its resolution

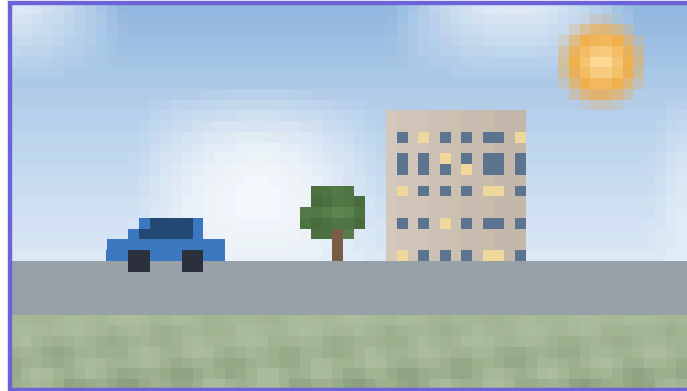
1920 × 1080



2,073,600 pixels

- one stored pixel, at this size

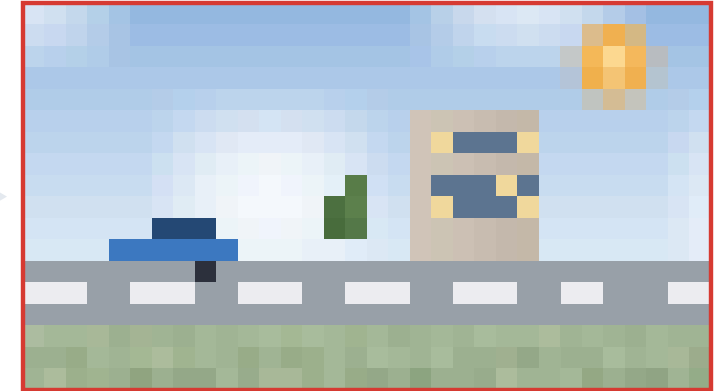
1280 × 720



921,600 pixels

- one stored pixel, at this size

640 × 360



230,400 pixels

- one stored pixel, at this size

all three are shown at the same size on the screen

fewer stored pixels, each stretched over more of the screen — the windows, the tree and the wheels go first

Storing the change instead of the whole picture was the first idea. The second is to make each picture out of fewer pixels: half the width and half the height leaves a quarter as many colours to write down.

Fewer pixels means fewer bits, and less detail on the screen.

1 Why video

2 Encoding

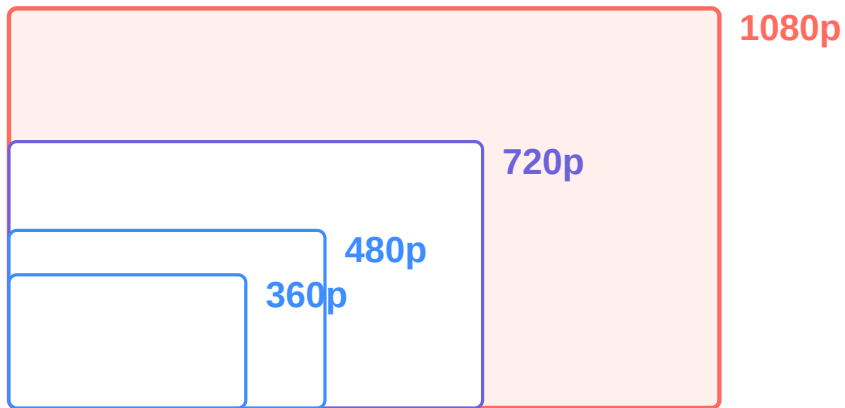
3 Segments

4 The player

5 Which row

6 CDN

Every resolution you have seen in a menu



drawn at their true relative size

1080p	1920 × 1080	2,073,600 px	
720p	1280 × 720	921,600 px	÷ 2.2
480p	854 × 480	409,920 px	÷ 5.1
360p	640 × 360	230,400 px	÷ 9.0

the number is the height of the picture in pixels

the p means progressive: every line is drawn in one pass

A 360p picture has one ninth the pixels of a 1080p one, so there is one ninth as much to write down. These are the names the same picture is offered under.

240p, 360p, 720p, 1080p, 4K are pixel counts, not a vague idea of quality.

1 Why video

2 Encoding

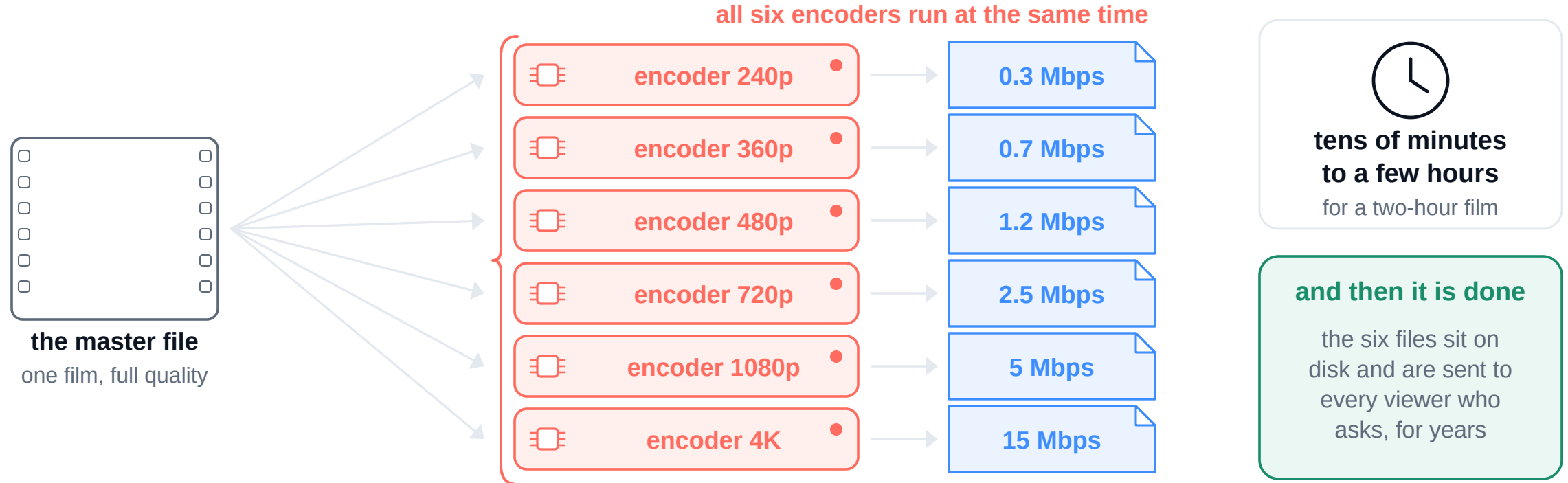
3 Segments

4 The player

5 Which row

6 CDN

One film goes in and six files come out



The six runs are independent, so they run at the same time on different machines. Encoding tries many ways to code every picture and is far slower than decoding, which has to keep up with playback on a phone.

All six versions exist before the first viewer asks for anything.

1 Why video

2 Encoding

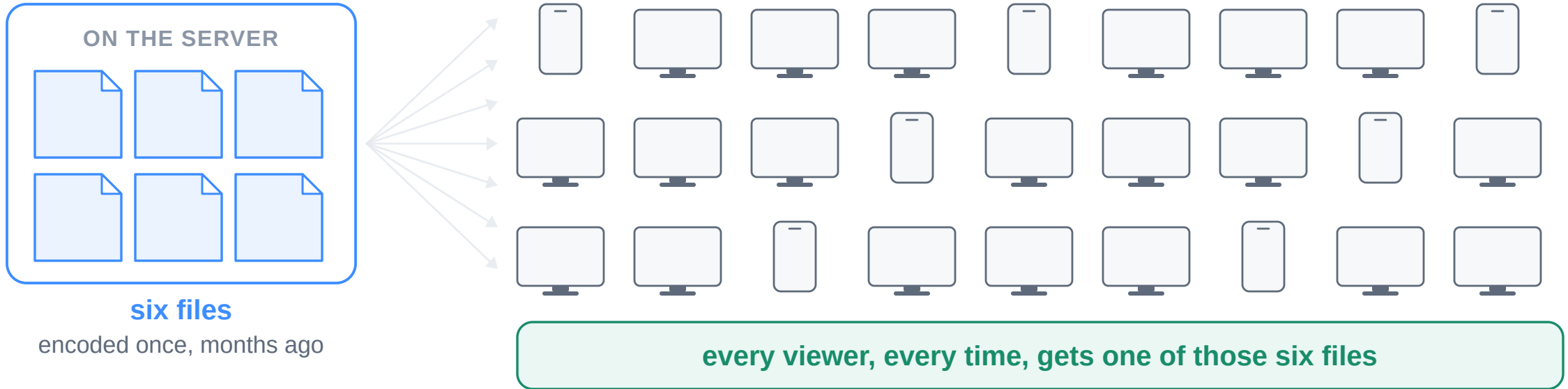
3 Segments

4 The player

5 Which row

6 CDN

The cost of encoding is paid one time



nobody encodes a film when a viewer presses play

it would take hours, and it would produce the same six files every time

A viewer can only ask for a version that already exists.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

These six rows are what the server holds

	NAME	RESOLUTION	BITRATE	ONE 4 s SEGMENT	
better ↑	4K	3840 × 2160	15.0 Mbps	7.50 MB	
	1080p	1920 × 1080	5.0 Mbps	2.50 MB	
	720p	1280 × 720	2.5 Mbps	1.25 MB	
	480p	854 × 480	1.2 Mbps	0.60 MB	
	360p	640 × 360	0.7 Mbps	0.35 MB	
	240p	426 × 240	0.3 Mbps	0.15 MB	

bitrate is how many bits of video there are in each second

This set is called the bitrate ladder. Bitrate × 4 seconds gives the size of one 4-second piece, which is what part 3 cuts the film into. Only one row is ever sent to one viewer.

Choosing a row is choosing how fast the network has to be. That is part 5.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

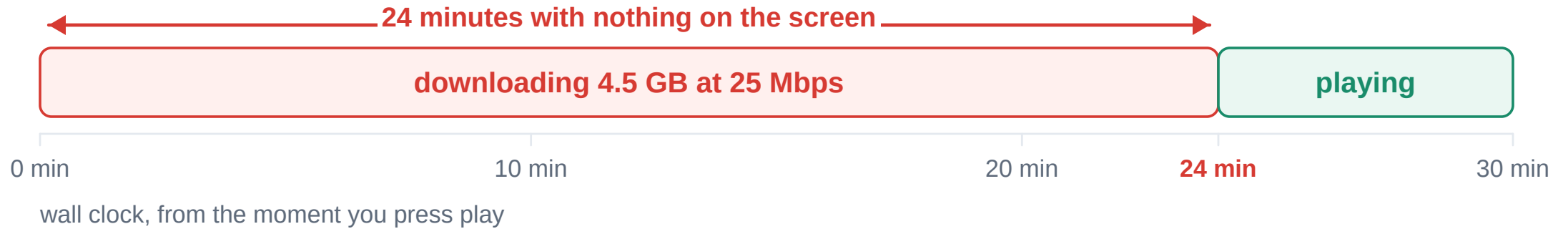
How is the encoded video handed to the player?

03

How it is cut up

Six encoded files are still six large files. Cutting them into short pieces, at the same instants, is what lets a player start early and change quality later.

Download the whole file, then play it



- ✗ the whole 4.5 GB has to be on your disk before the first frame
- ✗ stop watching after three minutes and you still moved 4.5 GB
- ✗ a phone often does not have 4.5 GB to spare

This is how a download manager works, and how video worked before broadband. The arithmetic is fixed: 4.5 GB is 36,000 Mb, and $36,000 \div 25$ is 1,440 seconds.

Playback cannot start until the last byte of the file has arrived.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

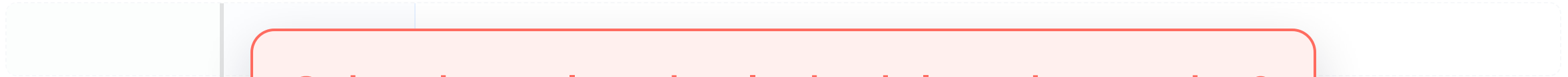
6 CDN

Download and play at the same time

☐ already played ☐ downloaded, not played yet ☐ not downloaded

a link that stays above the video

video in hand 15.2 s



So how does a player download and play at the same time?

the next slides are the answer

✓ gets 1.8 s of vid

a link that keeps dr

video in hand 0.5 s



STOPPED — waiting for video

✗ stopped, waiting for video — this has happened 1 time so far

the gap between the two marks is the video the player is holding — when it runs out, playback stops

As long as the download is faster than the playback, it never stops.

1 Why video

2 Encoding

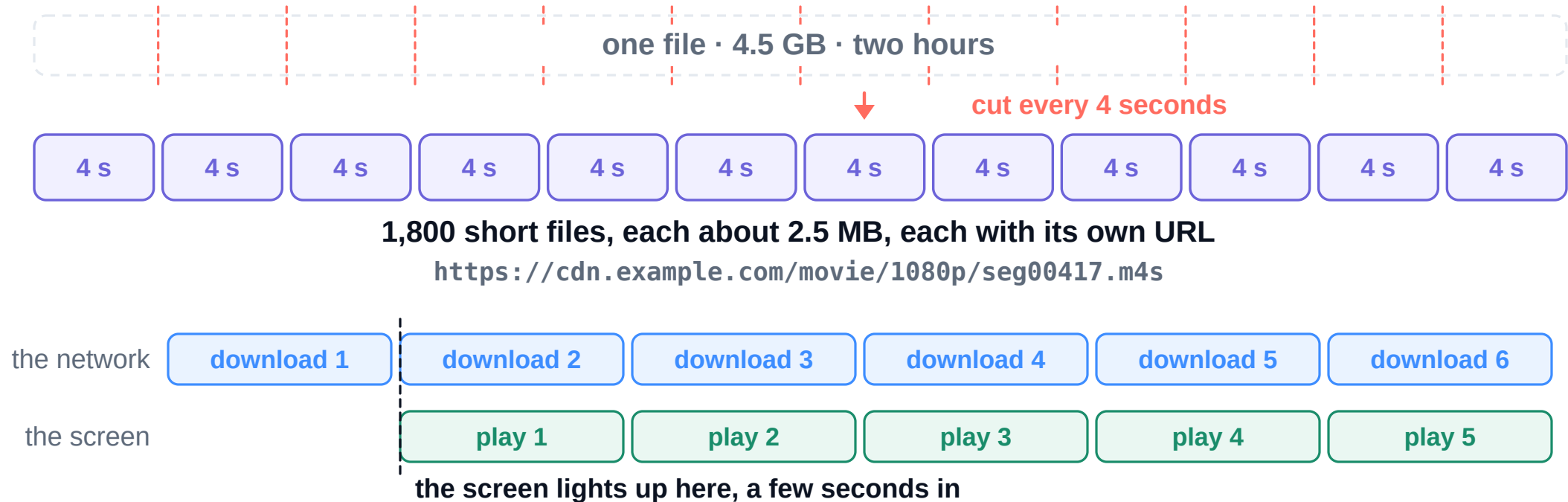
3 Segments

4 The player

5 Which row

6 CDN

Cut it into short files and fetch one at a time

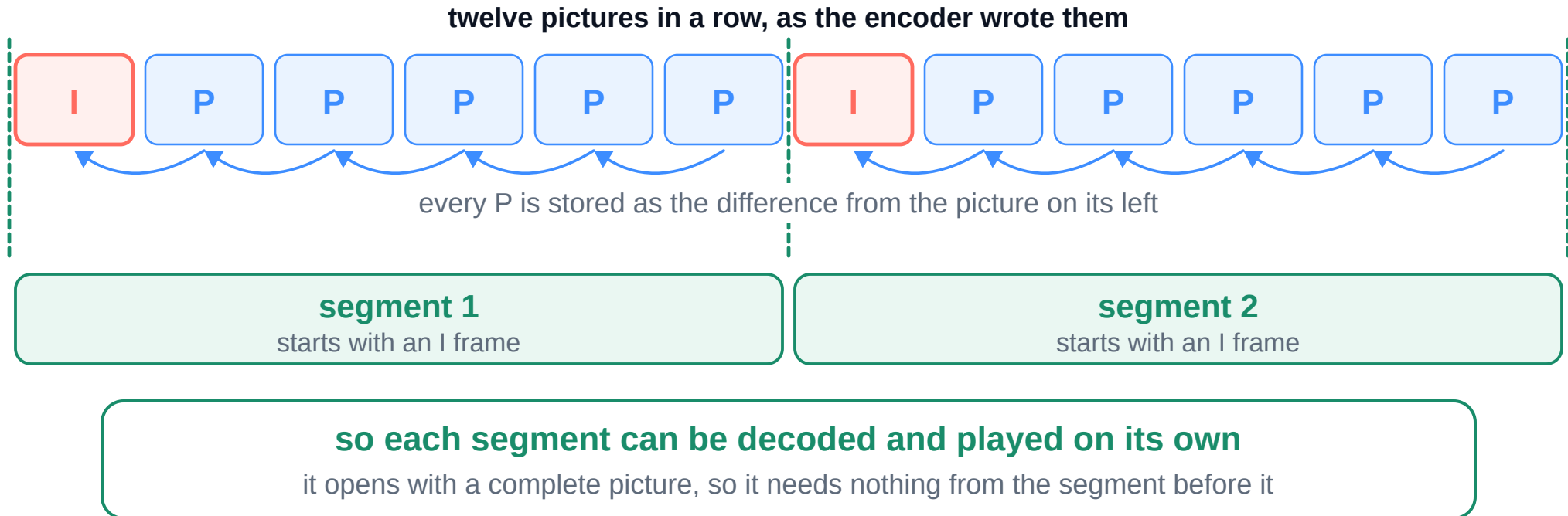


One file at one bitrate leaves the player nothing to do when the link slows down. Cutting the film up gives it something to work with, and the next download runs while the segment before it is on screen.

Playback starts after one segment, not after the whole film.



The video is cut at frames, on an I frame



An I frame is a complete picture and needs nothing before it. A P frame is only a difference, so a piece that started on one would be undecodable. The encoder is told to place an I frame every four seconds so the cuts have somewhere to land.

One segment in, one segment decoded. No segment depends on another.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

All six versions are cut at the same frames

frames	1–96	97–192	193–288	289–384	385–480	481–576	577–672	673–768
4K								
1080p								
720p								
480p								
360p								
240p								

every version is cut at the same frames

segment 417 means the same 96 frames of film in all six rows

One column of this table is one four-second piece of the film, six times.

1 Why video

2 Encoding

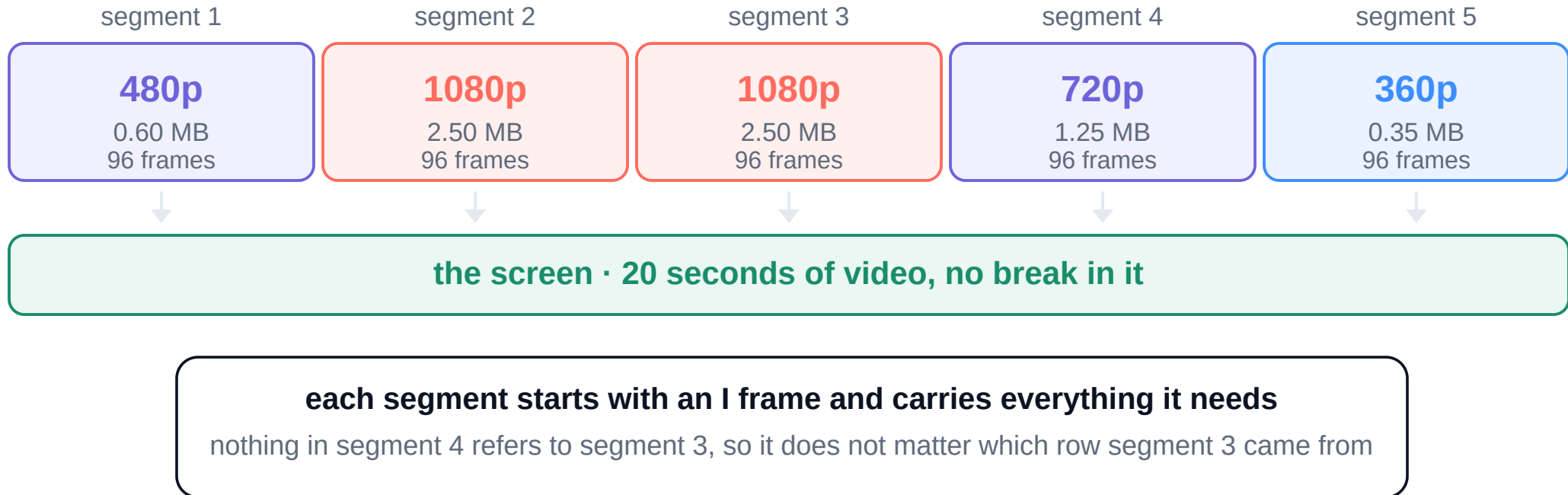
3 Segments

4 The player

5 Which row

6 CDN

Every segment is decoded on its own



The segments above are five different qualities, five different file sizes, and the same four seconds of film each. Played back to back they make one continuous twenty seconds.

So the player can change row at any segment boundary.

1 Why video

2 Encoding

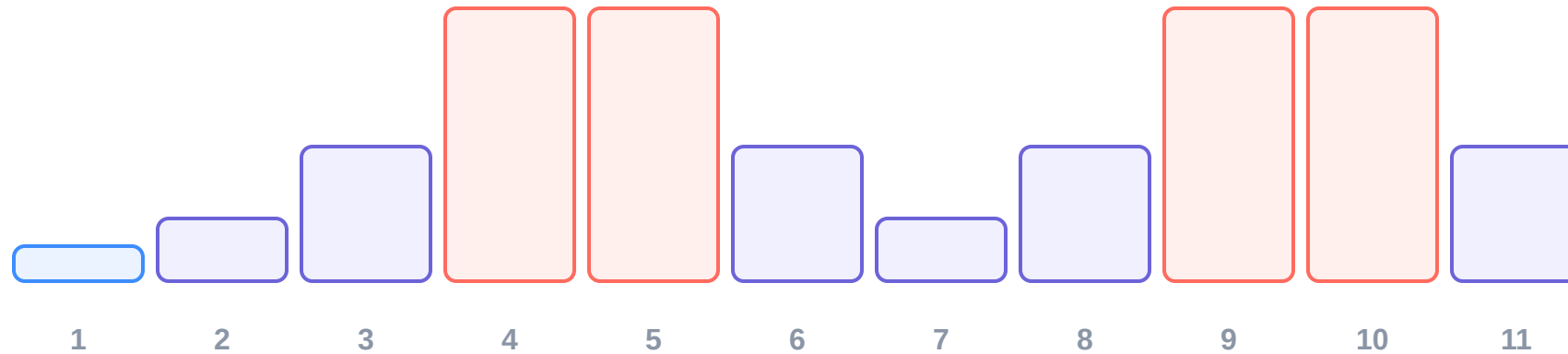
3 Segments

4 The player

5 Which row

6 CDN

One cell taken from every column, 1,800 times



the same eleven choices, taken out of the grid and laid out in time — the height is the bitrate

The rest of the lecture is walking across this grid and choosing a row.

1 Why video

2 Encoding

3 Segments

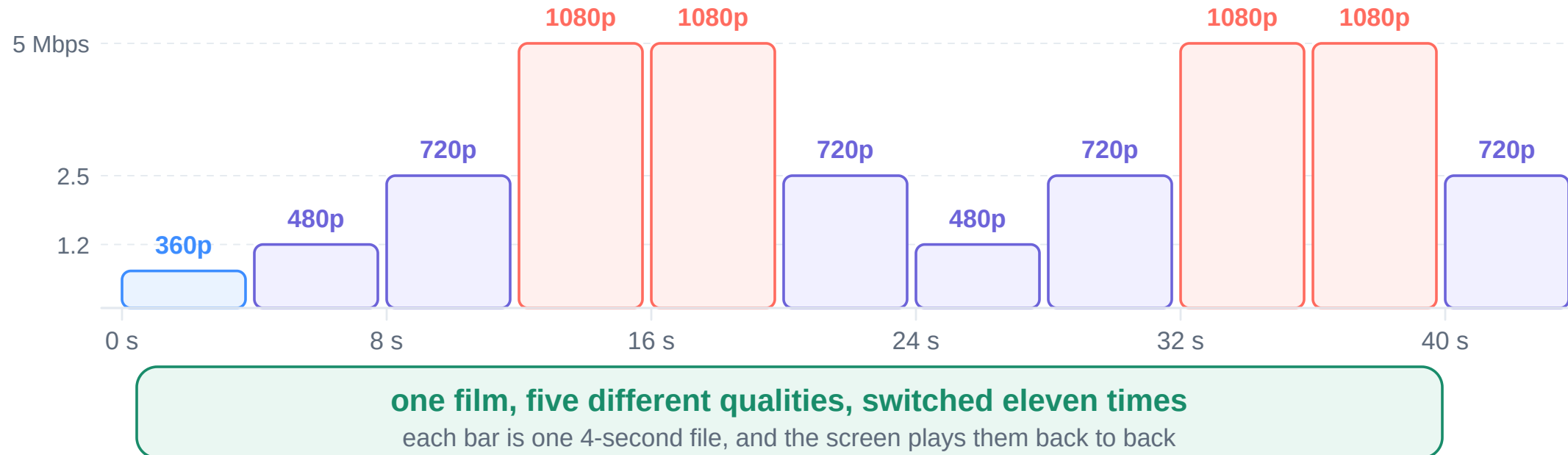
4 The player

5 Which row

6 CDN

The film you saw was never one quality

those eleven choices, laid out in the order they are played



What reaches the screen is a sequence of choices, not a single file.

1 Why video

2 Encoding

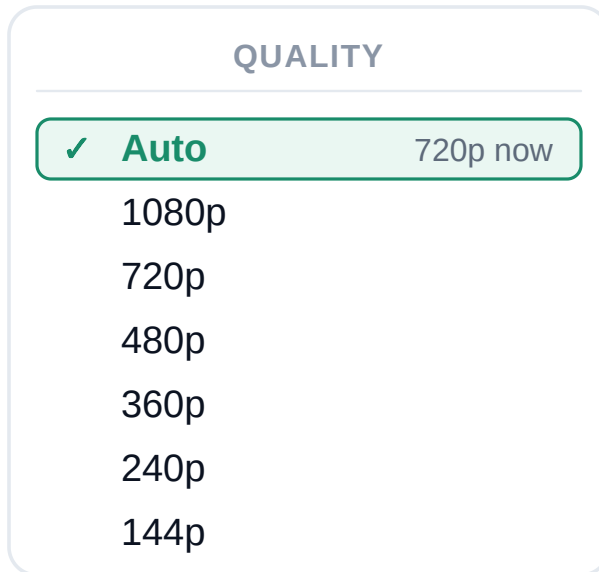
3 Segments

4 The player

5 Which row

6 CDN

Why a video player lets you fix the quality



every video player has this menu

Auto — the player picks a row for every segment

the eleven from the last slide



✓ the picture changes quality, but it never stops

1080p — you pick the row and the player stops choosing



✗ the link cannot hold 5 Mbps, so the buffer runs out

picking a number tells the player to ignore the network

On YouTube the setting is called Auto and it is the default: the player chooses a row for every segment. Pick a number instead and you have taken that decision away from it.

Fixing the quality removes the only move the player has when the link slows down.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

A small text file lists every segment and every quality

FIRST FILE: THE LIST OF QUALITIES

```
1 #EXTM3U
2 #EXT-X-STREAM-INF:BANDWIDTH=300000,RESOLUTION=426x240
3 240p/index.m3u8
4 #EXT-X-STREAM-INF:BANDWIDTH=1200000,RESOLUTION=854x480
5 480p/index.m3u8
6 #EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
7 1080p/index.m3u8
```

SECOND FILE: THE LIST OF SEGMENTS

```
1 #EXTM3U
2 #EXT-X-TARGETDURATION:4
3 #EXTINF:4.000,
4 seg00001.m4s
5 #EXTINF:4.000,
6 seg00002.m4s
7 #EXTINF:4.000,
8 seg00003.m4s
9 #EXT-X-ENDLIST
```

each line in the left file is the address of one file like the right one — one per quality

HLS calls it a playlist and uses **.m3u8**. DASH calls it a media presentation description and uses XML in an **.mpd** file. The player downloads it first, so it knows every URL it can request and the bitrate of each one.

The player reads the manifest, then requests segments one at a time.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

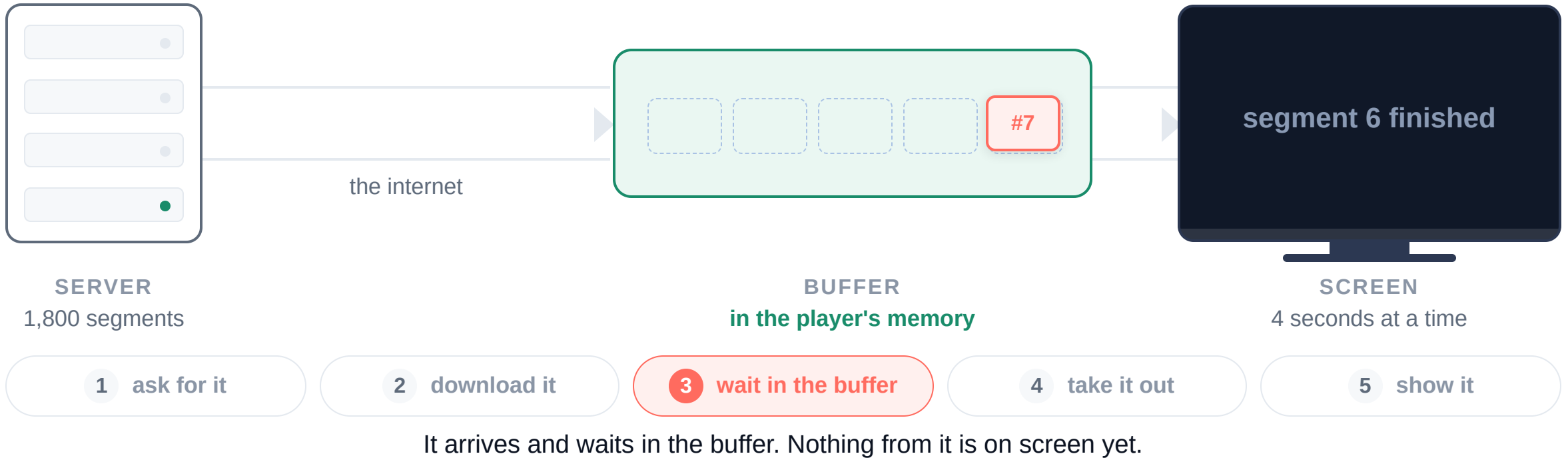
What does the player do with those files?

04

What the player does

It downloads segments one at a time, keeps a few of them ahead of the picture, and plays them in order. For this part, assume it always takes the same row of the ladder.

One segment, from the server to your screen



Download it, hold it in the buffer, then play it. Every segment does this.

1 Why video

2 Encoding

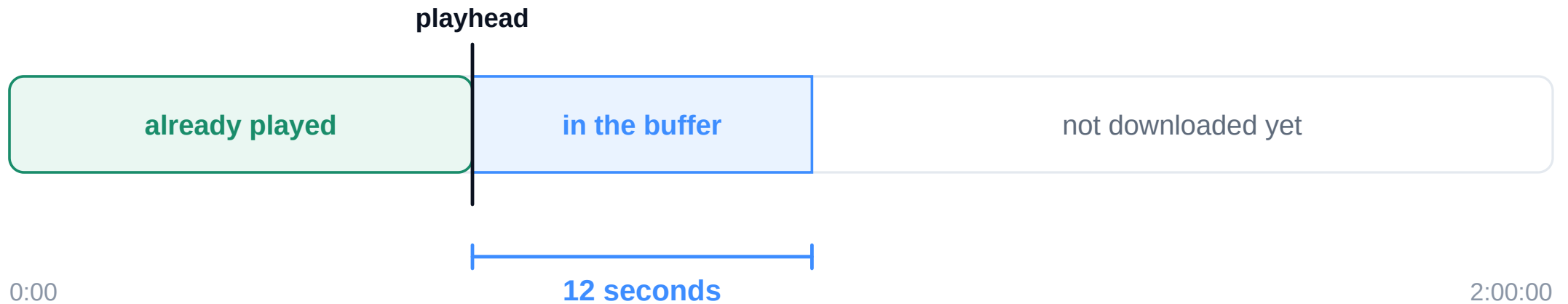
3 Segments

4 The player

5 Which row

6 CDN

The buffer is the video that is downloaded but not yet shown



$$\text{buffer} = \text{seconds downloaded} - \text{seconds played}$$

It is measured in seconds of video, not in bytes, because the deadline is in seconds. A player typically aims to hold between 30 and 240 seconds.

Buffer level is the number the player watches most closely.

1 Why video

2 Encoding

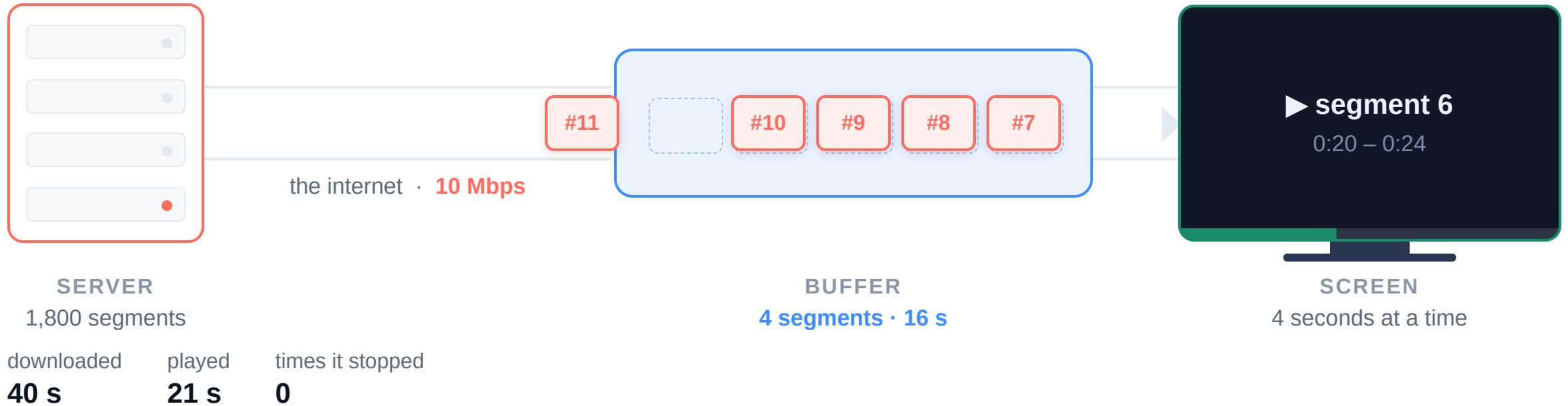
3 Segments

4 The player

5 Which row

6 CDN

The player does this over and over, without stopping



the video is 5 Mbps — set the network below it and the buffer empties

Set the network below 5 Mbps and watch the buffer empty until the video stops.

1 Why video

2 Encoding

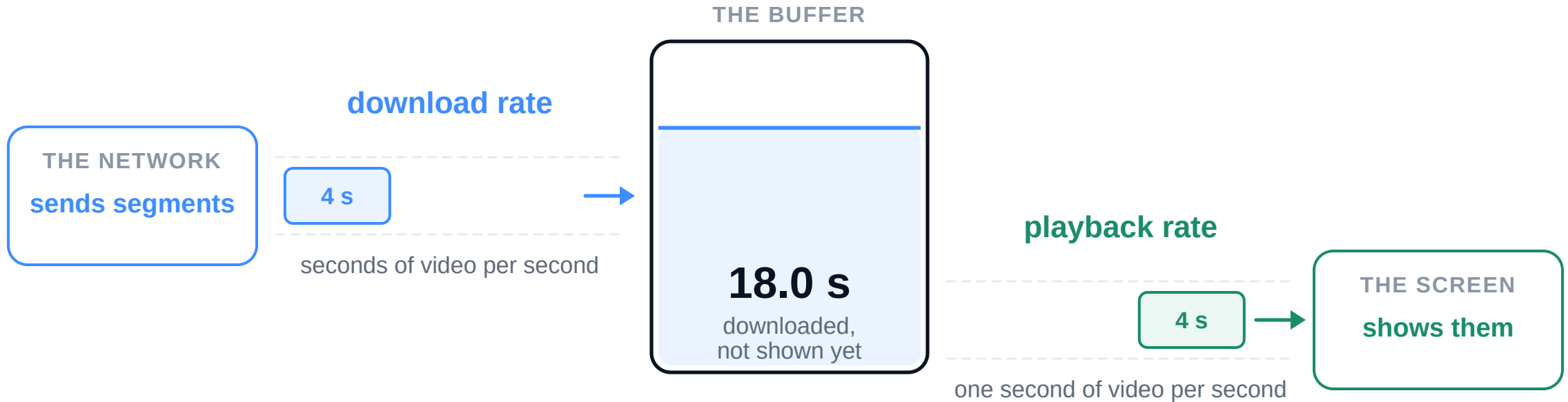
3 Segments

4 The player

5 Which row

6 CDN

The buffer sits between a download rate and a playback rate



segments arrive, wait in the buffer, and leave one every four seconds

The buffer is the video that has arrived but has not been shown yet.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

When the buffer is full, the player stops requesting



A player with a full buffer deliberately leaves the link idle.

1 Why video

2 Encoding

3 Segments

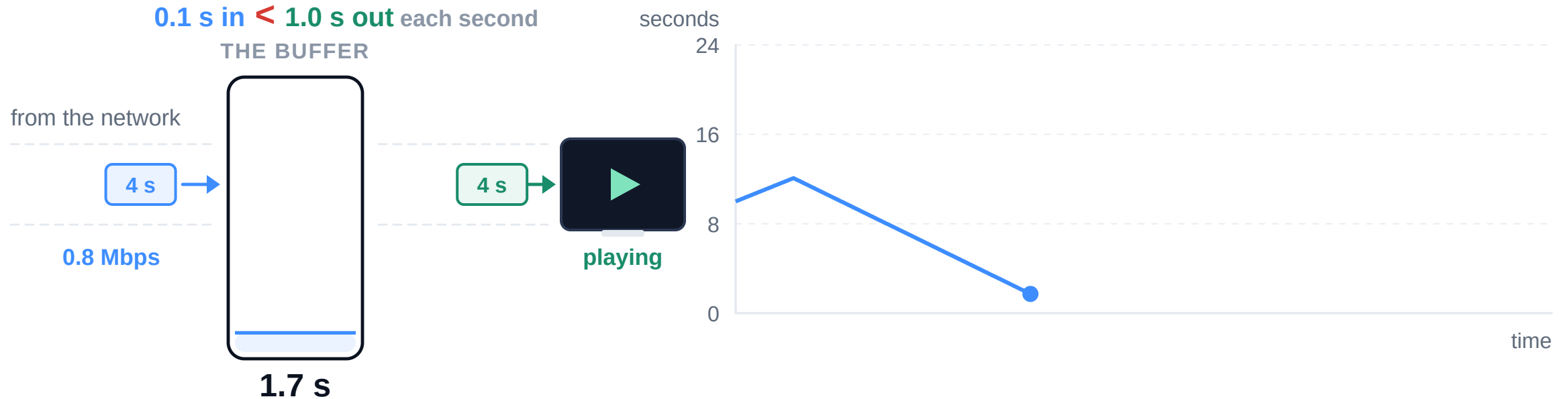
4 The player

5 Which row

6 CDN

● — Buffer empty

If the buffer reaches zero, playback stops until it refills



less goes in than comes out — the level falls by **0.8 s** every second

When the buffer reaches zero the picture freezes and waits.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

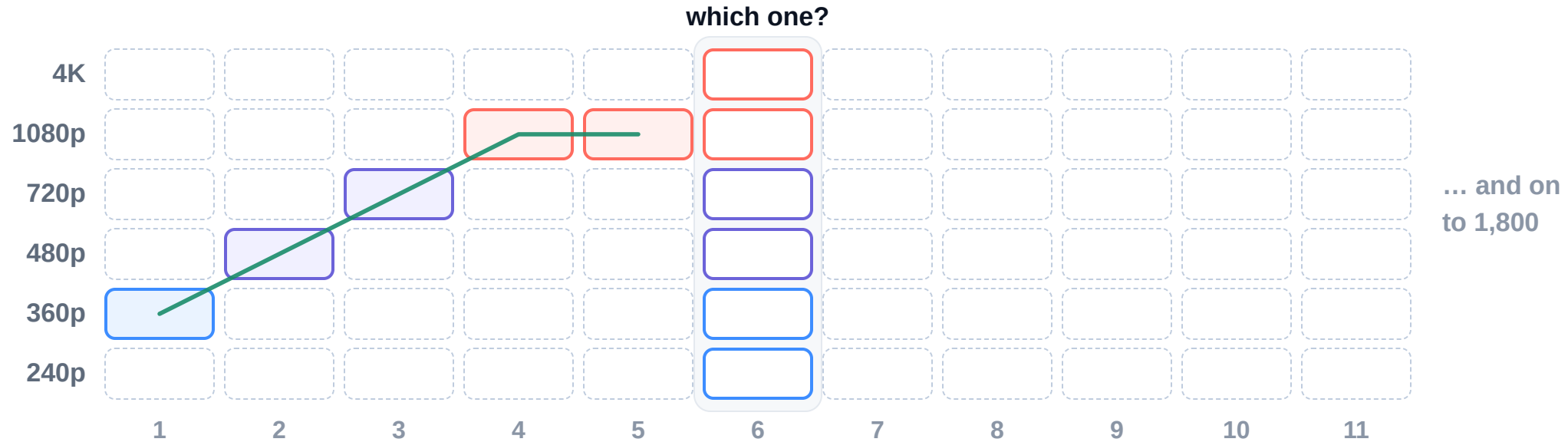
Which quality should the next segment be?

05

Which row of the ladder

The player has two measurements: how fast the network has been, and how much video is in the buffer. It turns them into one choice every four seconds. This is called adaptive bitrate selection, or ABR.

One cell taken from every column, 1,800 times



Segment 5 was taken at **1080p**. Segment 6 is next — and the network may have changed since.

Nothing stops the player changing row. The question is what to change to.

1 Why video

2 Encoding

3 Segments

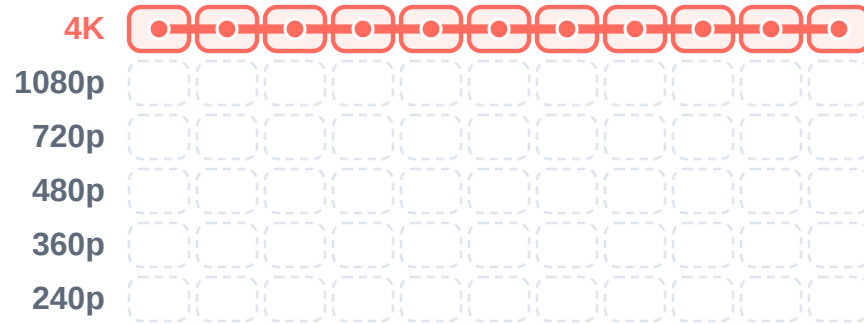
4 The player

5 Which row

6 CDN

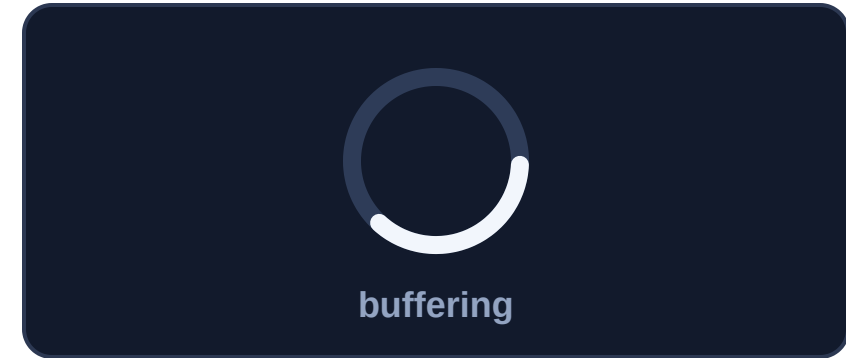
● The goal

Two things we want at the same time



every one of the 1,800 segments taken from the top row

high quality



the buffer hit zero and the picture stopped

no video freeze

The top row is 15 Mbps — 7.5 MB every four seconds. A link that cannot hold that rate empties the buffer, and the picture stops. So the two wishes pull against each other: the sharper the row, the sooner the video freezes.

Be as sharp as possible, and never stop. Every rule in part 5 is an attempt at both.

1 Why video

2 Encoding

3 Segments

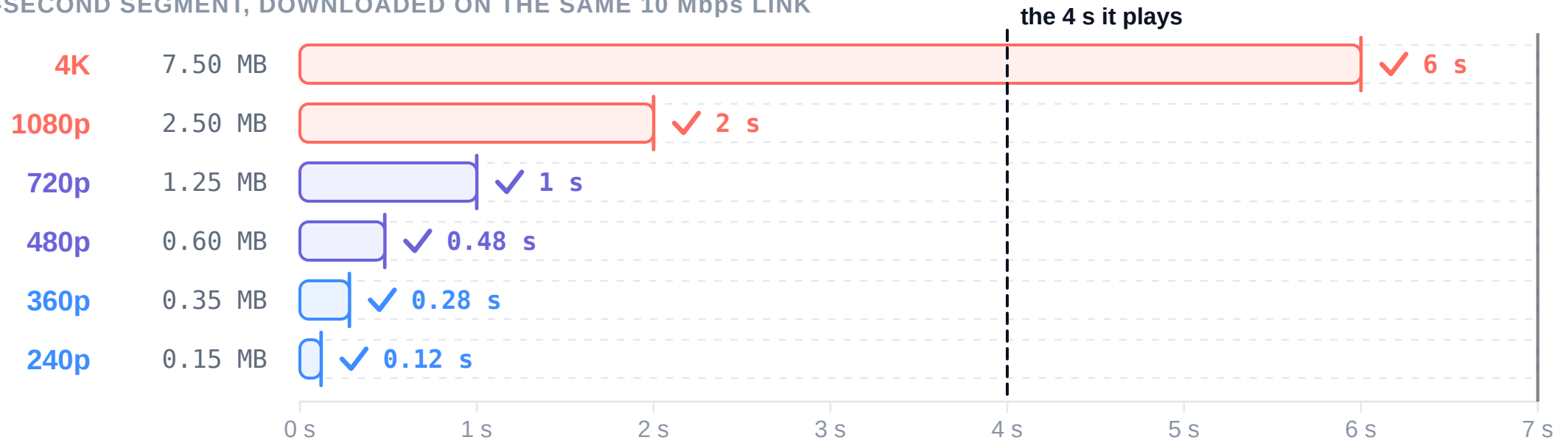
4 The player

5 Which row

6 CDN

On the same link, a higher quality takes longer

ONE 4-SECOND SEGMENT, DOWNLOADED ON THE SAME 10 Mbps LINK



the same four seconds of film every time — 4K took **50 times** as long as 240p, on the same link

4K is fifty times the bytes of 240p, so on one link it is fifty times the wait.

1 Why video

2 Encoding

3 Segments

4 The player

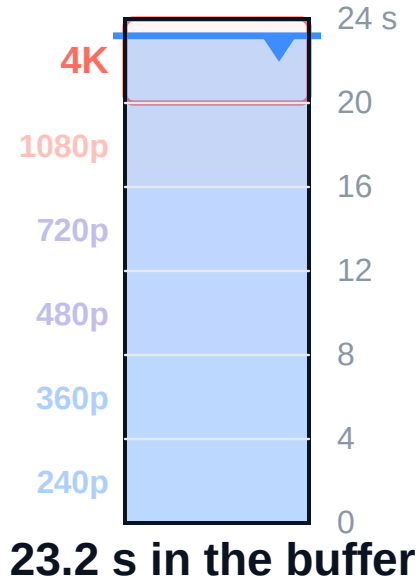
5 Which row

6 CDN

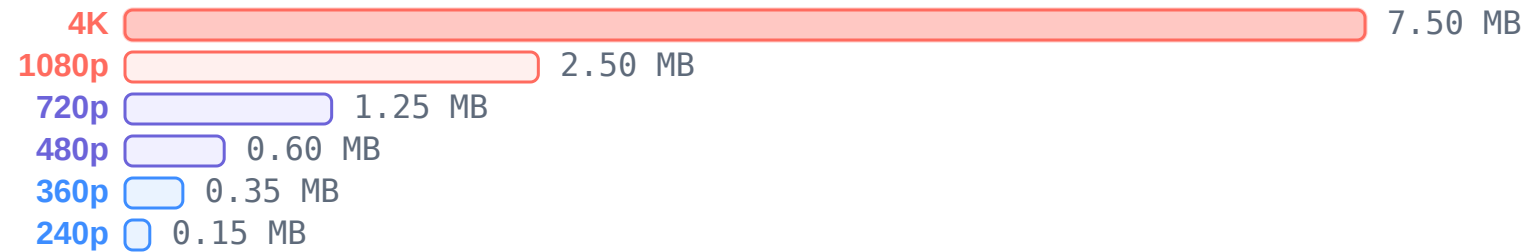
● Approach 1: watch the buffer

The fuller the buffer, the higher the quality

HOW MUCH VIDEO IS IN HAND

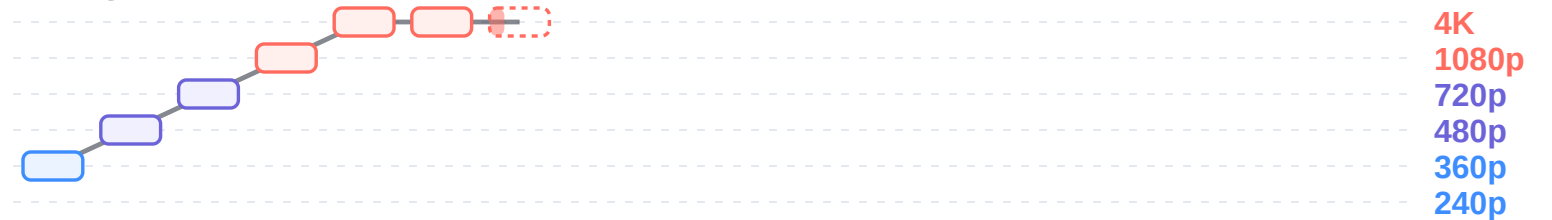


ONE 4-SECOND SEGMENT, AT EACH QUALITY



→ a bigger file takes longer to download

THE QUALITY THE PLAYER TOOK



the buffer is 23.2 s and falling → the 4K band, so the quality comes down

The whole rule: the level goes up, the quality goes up; the level drops, the quality drops.

1 Why video

2 Encoding

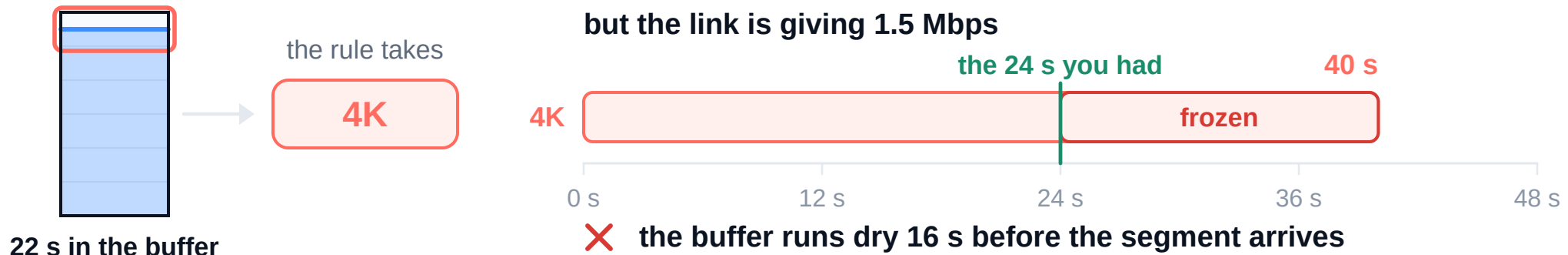
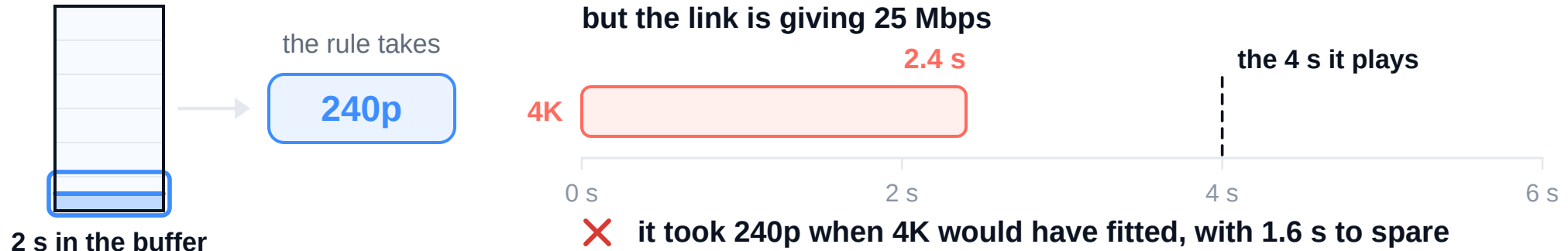
3 Segments

4 The player

5 Which row

6 CDN

The buffer level is not the download speed



the buffer says how much time you have — it says nothing about how fast the link is

Two signals, and the buffer is only one of them.

1 Why video

2 Encoding

3 Segments

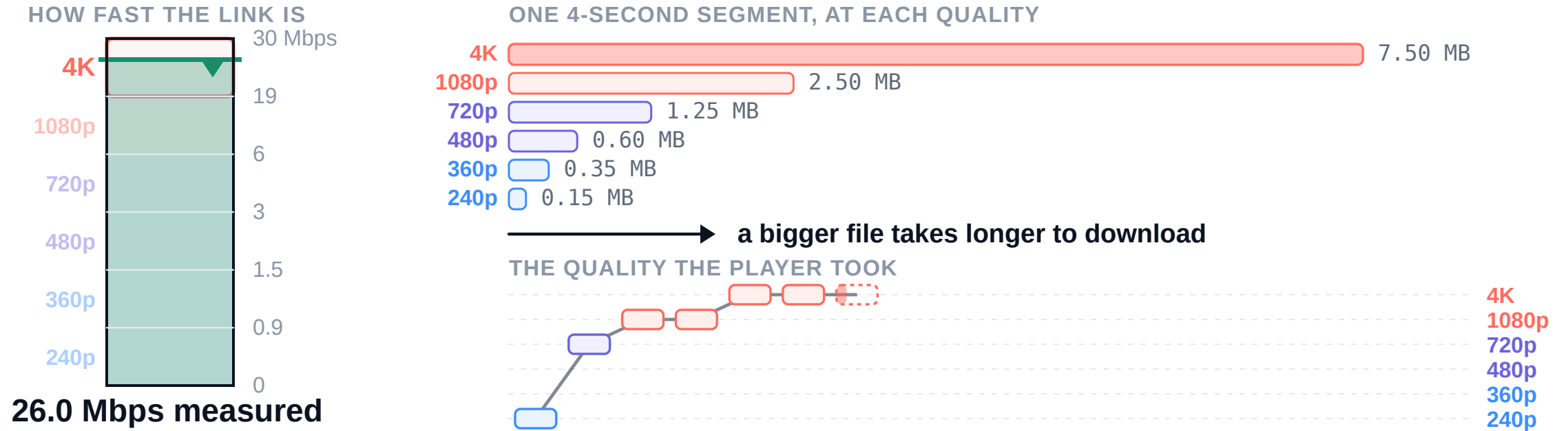
4 The player

5 Which row

6 CDN

● — Approach 2: watch the speed

The faster the link, the higher the quality



the link is measuring **26.0 Mbps** and falling → the **4K** band, so the quality comes down

Same picture, one signal swapped: the link speeds up, the quality goes up with it.

1 Why video

2 Encoding

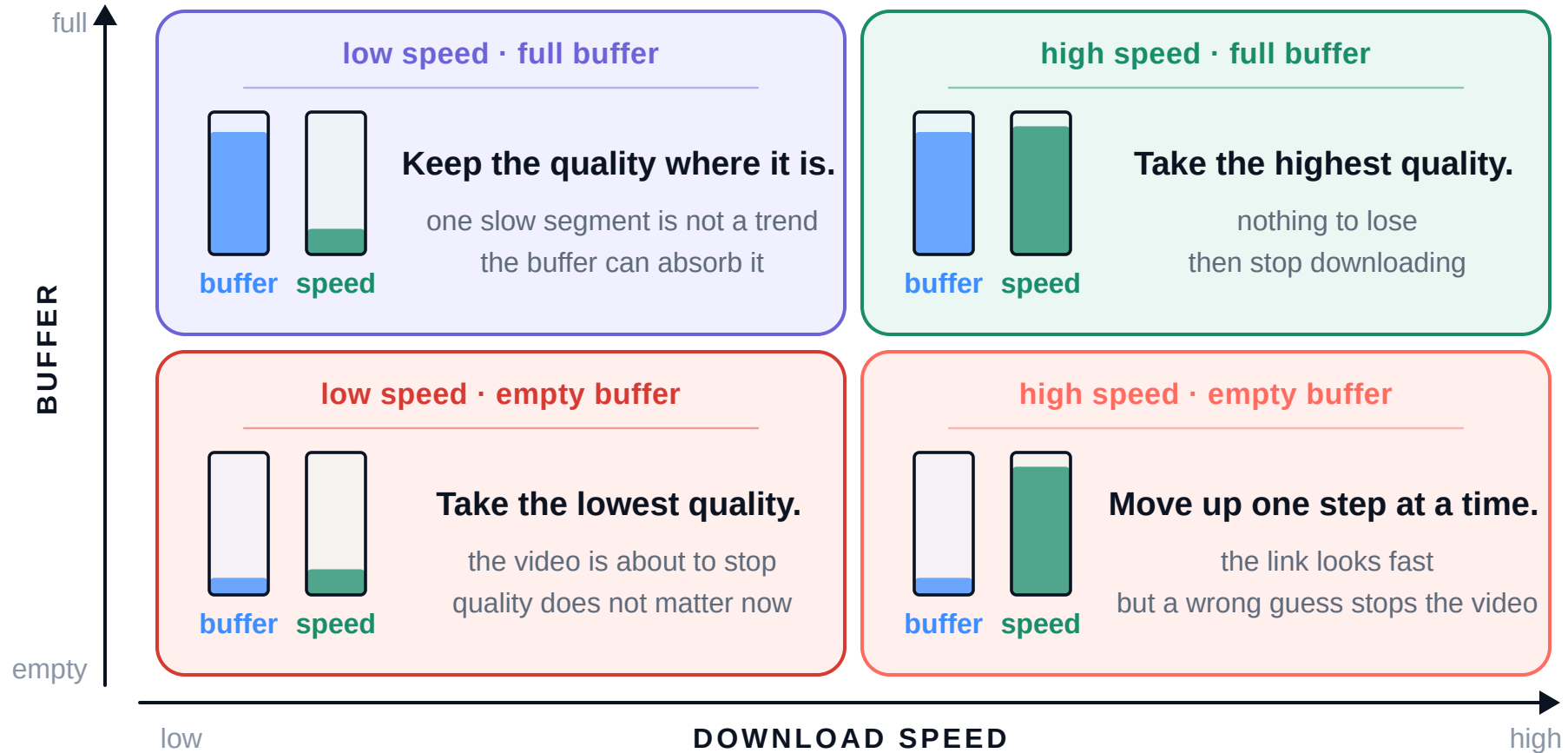
3 Segments

4 The player

5 Which row

6 CDN

Two signals give four situations and four answers



The two easy cases are the diagonals. The other two need both signals.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

One rule for each combination of the two signals

BUFFER	LINK SPEED	ACTION	REASON
empty	low	lowest quality	avoid stopping in the next few seconds
empty	high	move up slowly	build the buffer before using it
full	low	keep the quality	do not react to one slow segment
full	high	highest, then idle	take the quality and stop asking

a real player moves smoothly between these four cases rather than jumping

Two of the four rules refuse quality the connection could deliver. Being able to afford a row is not the same as it being safe to take it.

Every deployed algorithm is a smooth version of this table.

1 Why video

2 Encoding

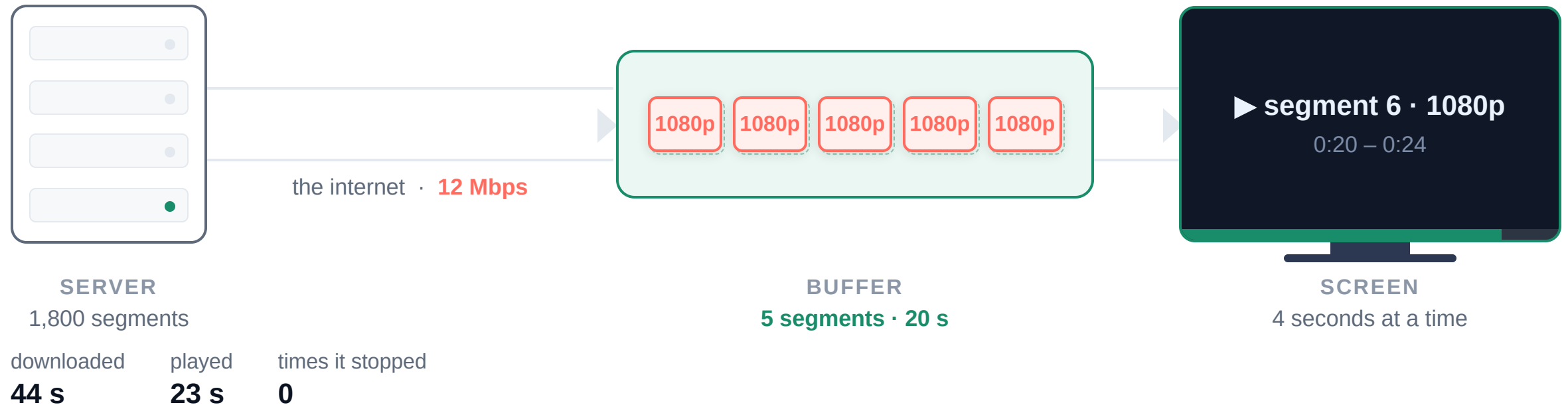
3 Segments

4 The player

5 Which row

6 CDN

The same loop, with the player choosing a quality



the colour of each block is the quality the player chose for it — what is on screen now was chosen about 20 seconds ago

The quality follows the network, and the buffer absorbs the short dips.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

They differ in which signal they trust

uses throughput only

estimate the throughput, take the highest row below it

FESTIVE · PANDA

uses the buffer only

a fixed function from buffer level to row

BBA 2014 · BOLA 2016

uses both, with a model

try the next few segments and keep the best sequence

MPC 2015

learns the rule from data

train on recorded network traces, no hand-written rule

Pensieve 2017

The first two are the two halves of the quadrant picture. The last two try to work out when each half is right. dash.js, the reference DASH player, ships BOLA and a combined rule.

Every commercial player is one of these four with extra tuning.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

Which machine sends you the segment?

06

Where the bytes come from

Part 3 assumed segments arrive quickly. That is arranged by a separate system: copies of the segments stored close to the viewer.

A request has to travel to the server and back



round-trip time is set by distance, and it does not get better
the player pays it once per segment, 1,800 times in a two-hour film

Every segment request, every quality switch and every seek waits for one round trip before any video arrives. On a 200 ms path that is 200 ms of doing nothing, 1,800 times.

Distance is a fixed cost that cannot be engineered away.

1 Why video

2 Encoding

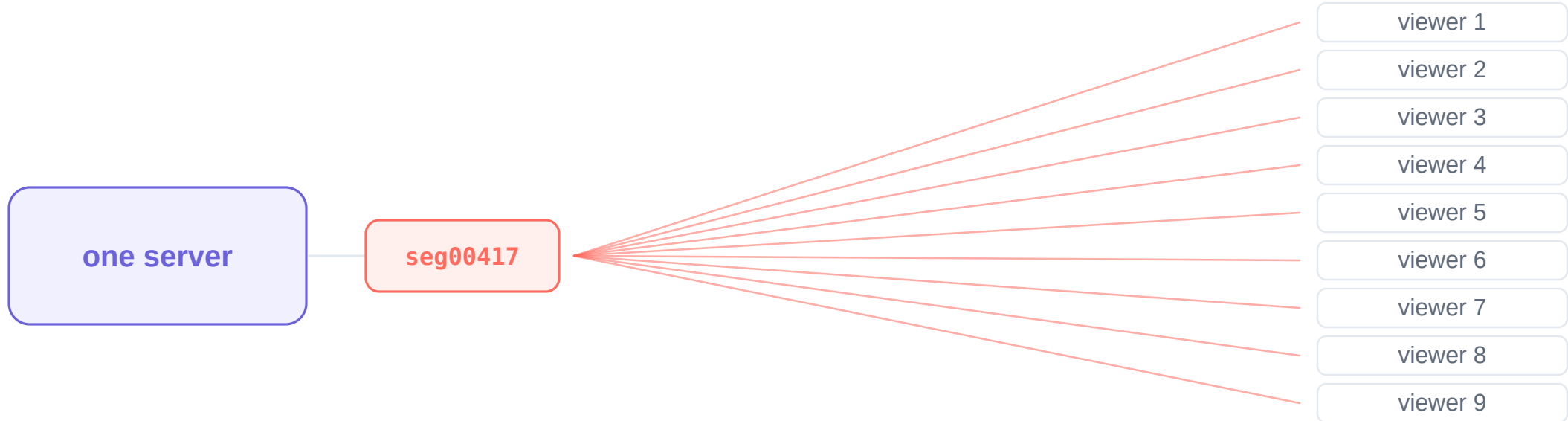
3 Segments

4 The player

5 Which row

6 CDN

Every viewer of a film asks for exactly the same files



the same bytes, nine times, over the same long path

a popular film is not nine viewers, it is millions

A small number of titles account for most requests, and every viewer of a title requests the same sequence of segment URLs in the same order.

The same bytes are sent over the same links again and again.

1 Why video

2 Encoding

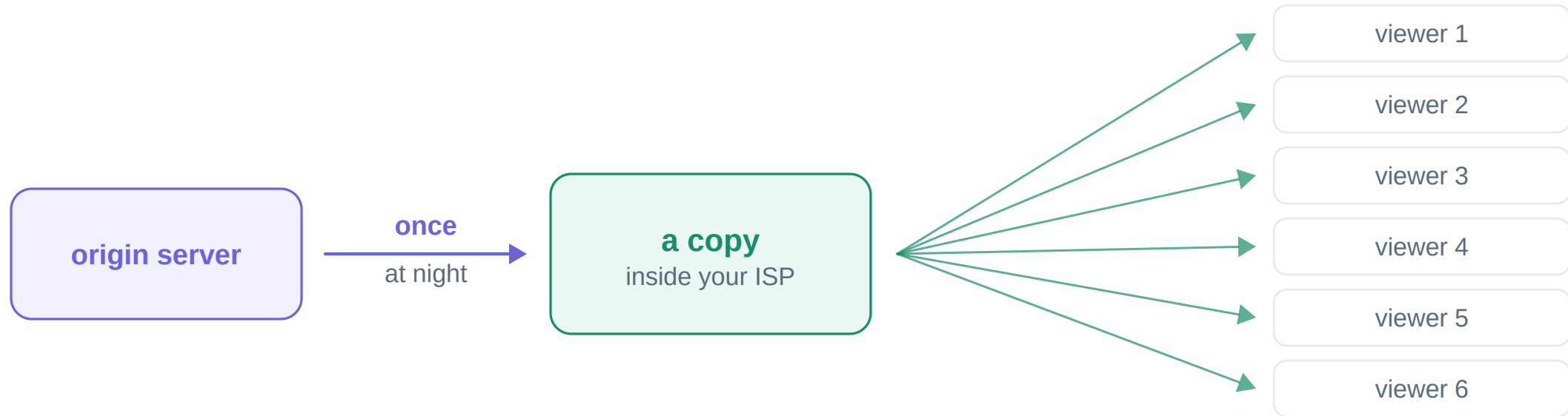
3 Segments

4 The player

5 Which row

6 CDN

Send the file over the long path once, then serve it locally



many times, over a short path

often in the same building as your broadband line

A content delivery network is a set of caches placed close to viewers, often inside the internet provider's own network. The long path is used once per title instead of once per viewer.

Latency drops, the origin server is no longer the limit, and the ISP pays less for transit.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

The URL stays the same; the machine behind it changes



the same name resolution you saw in the DNS lecture, used to pick a server

or: anycast

every copy announces the same IP address, and routing sends the packet to whichever one is nearest, with no per-viewer decision at all

either way, the URL in the manifest never changes

The manifest names segments by hostname and path. The CDN can point each viewer at a different server without the player, the manifest or the encoder knowing about it.

Naming and delivery are separate, so either can change on its own.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN

Netflix puts its own hardware inside internet providers

8,000+

appliances deployed

1,000+

partner ISPs

52+

internet exchange points

350 TB

of video per box

the boxes are filled at night, not when a viewer asks

Netflix predicts tomorrow's popular titles and copies them during off-peak hours

Netflix gives the hardware to qualifying providers for free, because the traffic it removes from their transit links is worth more than the boxes cost.

At this scale the CDN is not an optimisation; it is the delivery system.

1 Why video

2 Encoding

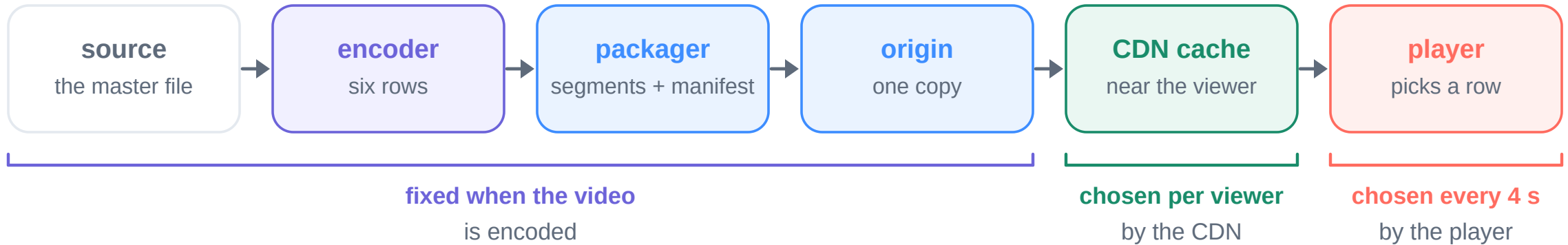
3 Segments

4 The player

5 Which row

6 CDN

Six stages, and three of them decide the quality



three decisions, on three different time scales, sharing one set of URLs

The encoder decides which qualities exist. The CDN decides which machine sends the bytes. The player decides which quality you get, and it is the only one that reacts to your connection.

The manifest is what connects the three.



Four questions that describe any streaming system

What is downloaded?

Segments of a few seconds, each a file with its own URL.

Who chooses the quality?

The player, from the manifest, once per segment.

What does it choose from?

Measured throughput and current buffer level.

Where do the bytes come from?

A cache near you, because segments are ordinary files.

They apply to YouTube, Twitch, a video doorbell and whatever comes next, because they name the four decisions any video delivery system has to make.

Cut the video into named files, and let the player choose which one to fetch.

1 Why video

2 Encoding

3 Segments

4 The player

5 Which row

6 CDN